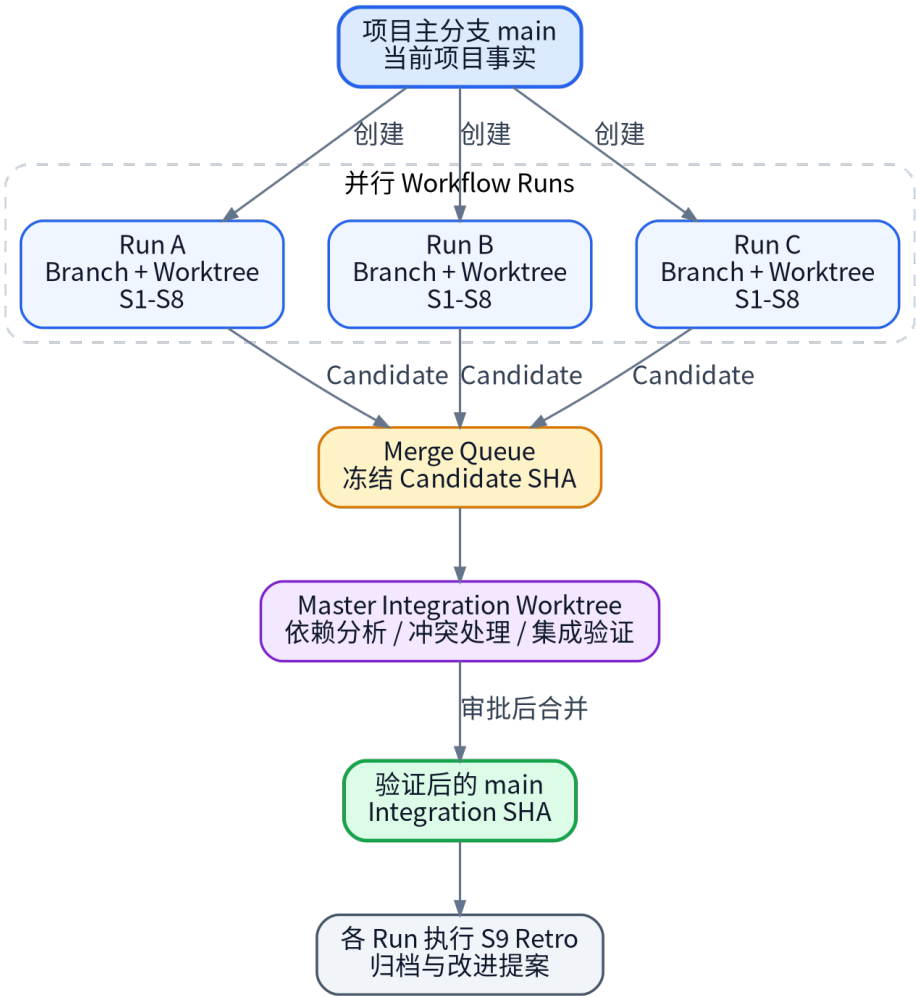


Prompt-Driven Parallel Workflow v2

Claude Code / Codex 多 workflow 协作教学手册



总览：每个目标独立开发，统一进入 Master Integration

本手册解决什么问题

如何让多个 Coding Agent 在同一项目中并行推进多个目标，同时保持状态可恢复、Agent 可接管、代码可验证、合并可审计。

版本 2.0 • 2026-07

阅读导航

章节	你会学到什么
1. 核心思想	为什么不能只依赖聊天历史，以及四种权威事实
2. 总体架构	Run、Branch、Worktree、Candidate 与 Integration 的关系
3. 九 State	从目标定义到终局复盘的完整推进与回退
4. 双 Agent 接管	Claude Code 与 Codex 如何在安全 Checkpoint 处切换
5. 并行 Run	多个目标如何隔离、依赖和冻结
6. Master Integration	如何处理文本、语义和架构冲突
7. 工件与证据	state、checkpoint、impact 和人类可读工件
8. 实战教程	第一次创建 Run、执行、接管、冻结和合并
9. 常见失败	错误用法、对抗性场景和恢复方式
10. 快速参考	审批 Token、检查表和常用 Prompt

建议阅读方式

第一次使用按顺序阅读；实际执行时，把第 8 章当操作教程，把第 10 章当速查表。

1. 从第一性原理理解这套工作流

Coding Agent 的核心困难并不是“不会写代码”，而是它在长任务中缺少稳定的外部状态、可靠反馈和明确的控制边界。聊天记录可以帮助理解，但不适合作为工程事实。

风险	常见表现	本工作流的解决方式
状态漂移	忘记当前 State、版本或下一步	state.json + checkpoint.json
范围漂移	未经批准扩大修改范围	Goal / Spec / Plan Gate
验证幻觉	未运行测试却宣称完成	命令输出与 SHA 绑定证据
Agent 切换失败	新 Agent 从头调查或误解进度	安全 Checkpoint 接管
并行污染	多个目标改在同一工作区	Branch + Worktree 隔离
合并盲区	Git 无冲突但行为冲突	Master 语义 Review + 集成验证

一句话原则

Prompt 负责引导，文件负责状态，Git 负责隔离，测试负责证明，人类负责关键门禁。

1.1 四种权威事实

- Project Truth：main 分支，代表项目当前已经接受的事实。
- Run Truth：workflow/<run-id> 分支及其 Run 工件，代表某个目标当前进展。
- Candidate Truth：冻结的 Candidate Commit SHA，代表被验证和 Review 的精确内容。
- Integration Truth：在集成 Worktree 中验证过的 Integration SHA，代表多个候选组合后的真实结果。



图 1：权威信息优先级 - 聊天历史只能作为最低层辅助信息

2. 总体架构：一个目标，一套隔离单元

最重要的结构不变量是：一个目标对应一个 Run；一个 Run 对应一个 Branch 和一个独立 Worktree。这样每个 Agent 都在自己的文件系统视图中工作，不需要在同一目录里频繁切换分支。

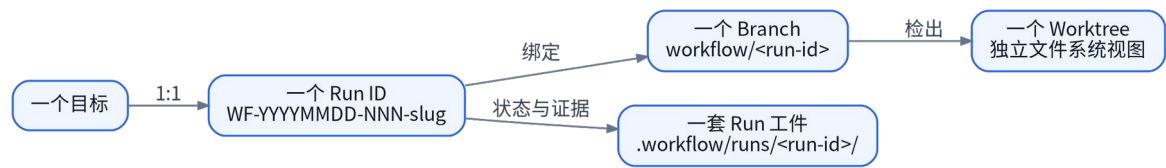


图 2：目标、Run、Branch、Worktree 和工件之间的 1:1 绑定

2.1 为什么必须使用 Worktree

- Branch 隔离 Git 历史，但 Worktree 进一步隔离当前文件、Index 和 HEAD。
- 多个 Agent 可以同时打开不同 Worktree，而不会直接覆盖彼此文件。
- 构建缓存和 IDE 索引仍可能是环境级问题，因此默认把 Worktree 放在项目同级目录。
- 每个 Agent 启动时都必须核验当前 Branch、Run ID、state.json 和 Worktree。

```
Run ID:   WF-20260725-001-auth-refresh
Branch:   workflow/WF-20260725-001-auth-refresh
Worktree: ../.workflow-worktrees/WF-20260725-001-auth-refresh
Artifacts: .workflow/runs/WF-20260725-001-auth-refresh/
```

2.2 共享区与 Run 区

区域	用途	谁可以写
.workflow/project/	稳定项目知识、架构、约定、命令	Master 或人工批准后更新
.workflow/control/	Registry、Merge Queue、Integration Log	Master Integrator
.workflow/runs/<run-id>/	当前 Run 的状态、工件和证据	当前 Run 的 Workflow Agent
.workflow/runtime/	本地锁、临时日志，不进入 Git	本地运行时

3. 九 State：从想法到可合并候选

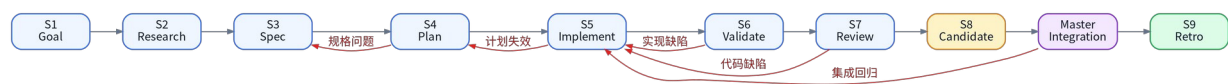


图 3：九 State 主路径以及典型回退边

不要把九 State 当成九个 Agent			
State 是任务阶段，不是模型角色。Claude Code 或 Codex 都可以执行任意合法 State。			
State	核心问题	主要工件	退出 Gate
S1 Goal	定义结果、范围、非目标和成功标准	goal.md	APPROVE GOAL
S2 Research	只读调查项目事实和证据	research.md	关键未知已解决
S3 Specification	形成可验证的需求契约	spec.md	验收标准可执行
S4 Planning	设计最小方案和原子步骤	plan.md + planned impact	APPROVE PLAN
S5 Implementation	按计划小步实现和局部验证	代码 + progress.md	批准步骤完成
S6 Validation	对照 Spec 证明正确性	validation.md	所有必须项通过
S7 Review	判断是否值得合入	review.md	无 Blocker
S8 Candidate	冻结精确 Commit SHA	candidate.md + actual impact	APPROVE CANDIDATE
S9 Retro	终局后复盘并提出改进	retro.md	Run 可归档

S1-S2：先确定方向，再理解项目

目标与事实是否足够清楚？

应该做	不应该做
- 把口语需求整理成 Goal Contract - 明确成功标准、范围和非目标 - 只读追踪相关代码、配置和测试 - 重要结论附路径或命令证据	- Goal 未批准就开始编码 - 把假设写成事实 - 无限制阅读整个仓库 - 让用户提供本可通过代码调查的信息
关键证据 Goal 版本审批 + Research 证据	

S3-S4：把理解变成可执行契约

什么算完成，以及如何最小化实现？

应该做	不应该做
- 每条验收标准可测试或可重复观察 - 区分需求与实现偏好 - 比较必要方案和权衡 - 把 Plan 拆成原子步骤并定义逐步验证	- 用“效果良好”等不可判定表述 - 在 Spec 中偷偷决定所有实现细节 - 计划外重构 - Plan 版本变化后沿用旧审批
关键证据 Spec 版本 + APPROVE PLAN <run-id> vN	

S5-S6：实现必须伴随外部反馈

代码是否按批准计划工作？

应该做	不应该做
- 每个 Step 先读现有代码 - 小步编辑并运行聚焦验证 - 更新 actual impact 和 Checkpoint - 完整映射 Acceptance Criteria	- 顺手优化无关代码 - 未运行命令却声称通过 - 同一失败策略无限重试 - 修改业务代码迎合错误测试
关键证据 Git Diff + Build/Test/Lint/Typecheck 输出	

S7-S8：Review 的对象必须不可变

被 Review 的内容是否就是将要合并的内容？

应该做	不应该做
-----	------

- Review 精确 SHA 和真实 Diff - 按 Blocker/High/Medium/Low 输出 Finding - 工作区干净后产生 Candidate Commit - 让 Validation、Review、Impact 绑定同一 SHA	- 只阅读实现者摘要而不看 Diff - 把风格偏好当缺陷 - Review 后继续修改却沿用旧证据 - Candidate 未批准就进入 Queue
关键证据 Reviewed SHA = Validated SHA = Candidate SHA	

S9：复盘真实终局

如何改进系统，而不是只总结功能？

应该做	不应该做
- 纳入接管、集成冲突和人工干预 - 区分一次性观察与可复用模式 - 输出改进提案和验证方法 - 记录最终 main SHA 或取消原因	- 在合并前过早复盘 - 自动修改全局规则 - 把一次失败固化为永久规定 - 只归责模型或用户而不分析系统原因
关键证据 merged / cancelled / terminal_failed 后执行	

4. Claude Code 与 Codex：对称执行与安全接管

两个 Coding Agent 共用同一协议，但不能共享完整会话。可迁移的是外部化状态，不可迁移的是聊天历史和内部推理。因此，切换必须发生在安全 Checkpoint。



图 4：标准跨 Agent 接管流程

可以共享	不能自动共享
Git Branch、HEAD、Diff	原会话聊天历史
state.json 和 checkpoint.json	私有推理、隐藏计划
Goal、Spec、Plan、审批	UI 中未写入文件的待办
测试与 Review 证据	产品私有 Memory
失败尝试和 next_action	未记录的人类决定

4.1 Checkpoint 的教学理解

Checkpoint 不是普通摘要，而是一份“新 Agent 不看旧聊天也能继续”的恢复契约。它必须回答：当前在哪、完成了什么、失败过什么、代码状态如何、下一步精确做什么。

```
{
  "run_id": "WF-...",
  "workflow_state": "S5_IMPLEMENTATION",
  "head_commit": "a83df17",
  "changed_files": ["src/auth/token.ts"],
  "verification": [{"command":"npm test -- token", "result":"12 passed"}],
  "failed_attempts": [{"approach":"reuse parser", "reason":"error semantics differ"}],
  "next_action": "Implement AC-04 and rerun focused tests."
}
```

默认建议

一个 Run 优先由一个 Agent 连续执行；只有额度、能力、停滞、独立视角或用户偏好需要时才切换。

5. Loop Engineering：循环必须能够收敛



图 5：内层执行、中层交付和外层学习三层 Loop

循环的价值来自反馈，而不是重复。每一轮必须产生新证据、改变策略或明确回退；否则就是无效消耗。

停滞信号	正确处理
相同或等价错误出现两次	停止同策略重试，换方案或回退
连续两轮失败项没有减少	重新检查假设和错误分类
Diff 在同一位置反复撤销	回到 Plan 或 Spec
同一命令无新假设地重复	记录 Plateau 并升级
需要突破批准范围	停止并重新审批
预算耗尽或高风险操作	请求人工决策

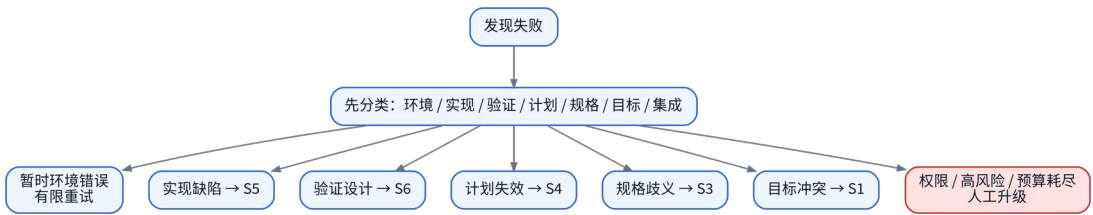


图 6：失败先分类，再选择正确回路

6. 工件体系：让状态脱离聊天窗口

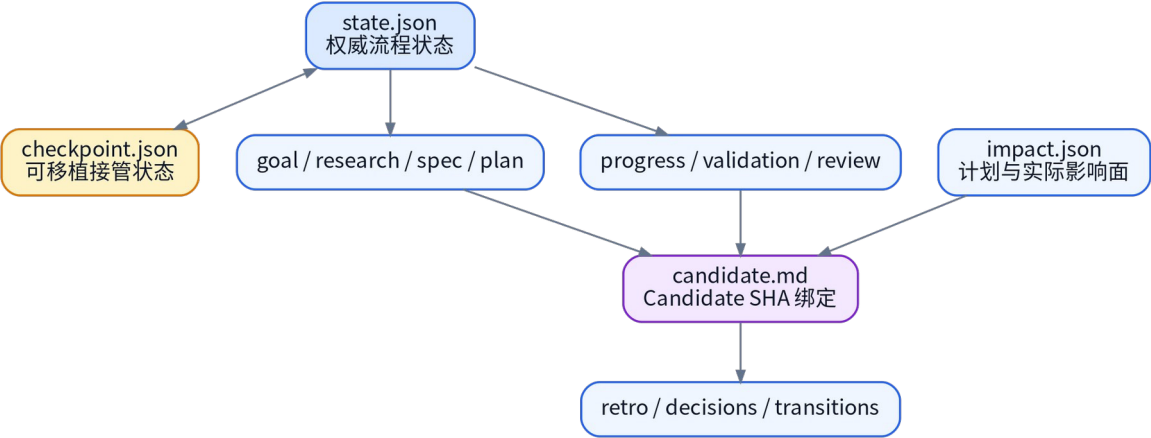


图 7: Run 工件之间的关系

工件	面向谁	核心用途
state.json	Agent / 工具	当前 State、版本、审批、预算、Candidate 和 next_action
checkpoint.json	接管 Agent	恢复当前步骤、Diff、验证、失败和下一动作
impact.json	Workflow Agent / Master	计划与实际影响面，发现跨 Run 语义冲突
goal/research/spec/plan	人和 Agent	目标、事实、契约和实施方案
progress/validation/review	人和 Agent	执行进度、正确性证据和质量判断
candidate.md	Master / 人	把所有证据绑定到精确 Candidate SHA
retro/decisions/transitions	项目治理	审批、状态变化和改进提案

6.1 为什么 SHA 绑定如此重要

如果 Review 通过后代码又发生变化，那么 Review 的对象和最终合并内容已经不同。Candidate Freeze 用 Commit SHA 消除这种“检查时和使用不一致”的风险。

失效规则

Candidate SHA 一旦变化，Candidate 审批、Validation 和 Review 全部失效，必须重新生成证据。

7. 多 Workflow 并行与 Master Integration

多个子 Run 可以并行推进，但它们不能自行决定如何进入 main。Master Integrator 是项目层角色，负责判断依赖、排序、同步、语义冲突和组合后的真实行为。

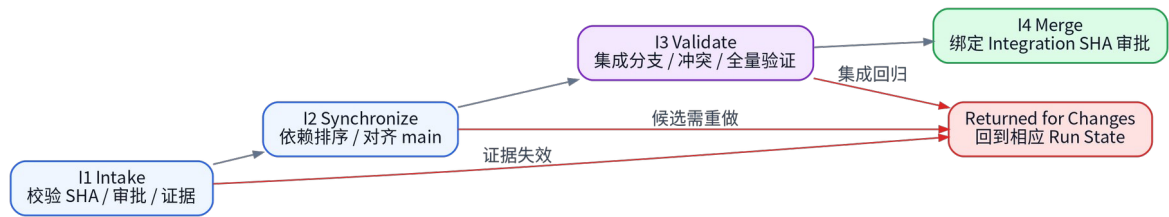


图 8：Master Integration 四阶段

冲突类型	Git 是否一定发现	Master 如何检查
文本冲突	通常可以	解决冲突并核对双方 Spec
API 语义冲突	不一定	比较接口行为和调用者假设
数据模型冲突	不一定	检查 Schema、Migration 和兼容性
配置/依赖冲突	不一定	检查配置键、版本和运行环境
架构方向冲突	不会	对照 Architecture 和两个 Plan
组合行为回归	不会	集成分支运行全量相关验证

7.1 为什么不能直接在 main 上解决冲突

主分支应该始终保持可构建、可恢复。所有试验性合并、Rebase 和冲突处理都应发生在独立 integration/Worktree。只有精确 Integration SHA 被验证并批准后，才进入 main。

```
I1 Intake
-> I2 Dependency & Synchronization
-> I3 Integration Branch + Full Validation
-> APPROVE INTEGRATION <integration-id> <integration-sha>
-> I4 Merge and Record
```

8. 第一次本地实验：完整操作教程

步骤 1：把工作流放入 Git 项目

1. 将 V2 ZIP 解压到现有 Git 仓库根目录。
2. 填写 .workflow/project/ 下的 CONTEXT、ARCHITECTURE、CONVENTIONS 和 COMMANDS。
3. 检查 .claude/settings.json 已关闭 Auto Memory。
4. 把工作流框架提交到 main。

步骤 2：运行健康检查

```
pwsh .workflow/tools/workflow.ps1 doctor
```

步骤 3：创建一个新 Run

```
pwsh .workflow/tools/workflow.ps1 new-run `
  -Title "实现 Refresh Token Rotation" `
  -Slug "refresh-token-rotation"
```

脚本会注册 Run、创建 Branch 和独立 Worktree，并输出需要在 Claude Code Desktop 或 Codex 中打开的目录。

步骤 4：在 Worktree 中对 Agent 说第一句话

开始当前 Worktree 对应的 Workflow。

请读取入口规则文件，根据当前 Git Branch 确定 Run ID，读取 state.json、checkpoint.json、当前 State 规则和必需工件。

先验证 Branch、Run ID、State 和 Worktree 是否一致，然后从 next_action 继续。

如果这是新 Run，请从 S1 Goal 开始，不要立即修改业务代码。

步骤 5：按 Gate 批准

```
APPROVE GOAL <run-id> v1
APPROVE PLAN <run-id> v1
APPROVE CANDIDATE <run-id> <candidate-sha>
```

步骤 6：需要切换 Agent 时

```
# Agent A
pwsh .workflow/tools/workflow.ps1 checkpoint `
  -Agent claude `
  -NextAction "Implement AC-04 and rerun focused tests."
pwsh .workflow/tools/workflow.ps1 release -Agent claude

# Agent B
pwsh .workflow/tools/workflow.ps1 acquire -Agent codex
```

接管提示词

接管当前 Workflow。使用新会话读取入口规则、state.json、checkpoint.json、当前 State 和 Git Diff。验证 Branch/Run/HEAD 一致，获取本地写锁后，从 next_action 继续。不要重新启动整个 Workflow。

步骤 7：冻结 Candidate

5. 确保 S6 Validation 通过，S7 Review 无 Blocker。
6. 提交当前实现并确保工作区干净。
7. 进入 S8，执行 freeze，记录 Candidate SHA。
8. 用户批准精确 SHA 后进入 Merge Queue。

步骤 8：Master Integration

9. Master 读取 Queue 中每个 Candidate 的 state、impact、validation、review 和 Diff。
10. 创建独立 Integration Worktree。
11. 按照依赖顺序合并，处理文本和语义冲突。
12. 运行组合后的完整验证。
13. 用户批准精确 Integration SHA 后合并 main。
14. 相关 Run 进入 S9 Retro。

9. 对抗性审查：常见错误与恢复

错误场景	为什么危险	正确动作
Agent 在错误 Worktree 工作	可能把一个目标改进另一个 Branch	Branch-Run-State 三方校验，立刻停止
两个 Agent 同时写同一 Worktree	文件覆盖与状态竞争	单写者 Lock；第二个 Agent 只读
Agent 未写 Checkpoint 就退出	接管者不知道未记录状态	Recovery Mode 重建事实
Plan v2 修改后沿用 v1 审批	执行内容未经人类批准	旧审批失效，重新批准
Review 后继续修改 Candidate	Review 与合并对象不一致	SHA 变化后重新 S6-S8
Git 自动合并后直接进 main	可能存在语义冲突	Integration Worktree 全量验证
子 Run 修改项目长期规则	并行 Branch 会冲突且规则未经验证	输出 Project Update Proposal
依赖聊天历史继续	换 Agent/上下文后信息丢失	重要事实写入工件

最重要的恢复原则

遇到不确定性时，先停止写入，回到 Git、State、Checkpoint 和外部证据重建事实，而不是根据聊天印象继续。

10. 快速参考

10.1 批 Token

```
APPROVE GOAL <run-id> vN
APPROVE PLAN <run-id> vN
APPROVE CANDIDATE <run-id> <candidate-sha>
APPROVE INTEGRATION <integration-id> <integration-sha>
ACCEPT RISK <run-id> <risk-id>
AUTHORIZE ACTION <run-id> <action-id>
FORCE TAKEOVER <run-id> BY <agent>
```

10.2 Agent 开始工作前

- 确认当前 Git Branch 与 Run ID。
- 确认 state.json 的 Branch、State 和版本。
- 读取 checkpoint.json 的 next_action。
- 检查审批是否绑定当前版本或 SHA。
- 确认写锁和当前 State 所需能力。
- 检查 Git Status 与真实 Diff。

10.3 Agent 结束工作前

- 更新代码和所有相关工作件。
- 记录真实命令、结果和未运行项。
- 更新 actual impact。
- 写入安全 Checkpoint。
- 明确下一动作、开放风险和是否需要审批。
- 需要切换时释放写锁。

10.4 十二条核心不变量

1. 一个目标对应一个 Run。	7. 子 Run 不修改全局 Control。
2. 一个 Run 对应一个 Branch。	8. 子 Run 不直接合并 main。
3. 一个 Branch 对应一个 Worktree。	9. Candidate SHA 冻结后不可变。
4. Branch 名决定 Run ID。	10. SHA 变化使证据和审批失效。
5. 同一 Worktree 同时只有一个写入 Agent。	11. 所有集成都在独立 Integration Worktree 完成。
6. Agent 切换只发生在安全 Checkpoint。	12. S9 在终局结果之后执行。

附录 A：目录速览

AGENTS.md	# 跨 Agent 入口
CLAUDE.md	# Claude 入口并导入 AGENTS.md
.workflow/	
core/	# 核心协议
states/	# S1-S9
project/	# 稳定项目知识
runs/<run-id>/	# 独立 Run 工件
control/	# Registry / Merge Queue
integration/	# I1-I4
runtime/	# 本地锁, Git 忽略
tools/workflow.ps1	# 创建、锁、Checkpoint、冻结、入队

附录 B：人与 Agent 的职责边界

参与者	主要职责	不能替代什么
Workflow Agent	执行当前 Run 的合法 State、维护证据与 Checkpoint	不能自行批准或直接合并 main
Master Integrator	跨 Run 依赖、冲突和集成验证	不能悄悄重写子 Run 目标
人类 Owner	批准 Goal、Plan、Candidate、Integration 和高风险动作	不需要人工维护所有状态文件
Git/测试工具	隔离版本、提供外部反馈	不能理解业务语义和价值取舍